RAG

Mikel Egaña Aranguren

mikel-egana-aranguren.github.io

mikel.egana@ehu.eus

Universidad del País Vasco
Euskal Herriko Unibertsitatea

BILBOKO INGENIARITZA ESKOLA
ESCUELA DE INGENIERÍA DE BILBAO

RAG

https://github.com/mikel-egana-aranguren/ABD

CC BY

¿Qué es RAG?

RAG: Retrieval-Augmented Generation

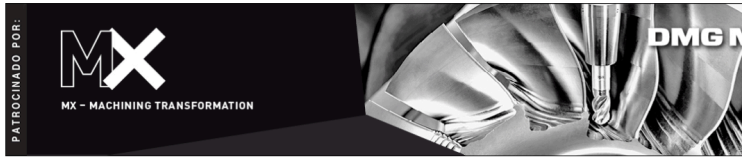
Técnica para mejorar respuestas de LLMs mediante recuperación de información externa

Combina: búsqueda de información (Retrieval) + generación de texto (Generation)

Objetivo: respuestas más precisas, actualizadas y verificables

RAG CFAA

Interempresas



TECNOLOGÍA

Esta técnica combina la capacidad generativa de los modelos de lenguaje a gran escala o LLMs con la recuperación de información externa y verificable

RAG industrial: cuando las máquinas responden a preguntas

Nahia Iturregi Artiñano y Endika Tapia Fernández, investigadores en el CFAA; Mikel Egaña Aranguren y Unai López Novoa, profesores agregados en el departamento de Lenguajes y Sistemas Informáticos (LSI) de la EHU; y Leonardo Sastoque Pinilla, investigador doctor del CFAA, coordinador de proyectos de I+D y digitalización industrial. 12/11/2025

1270

En este artículo se presenta una introducción a los fundamentos de la técnica RAG y su aplicación en la mejora de la precisión y relevancia de las respuestas generadas por modelos de lenguaje. Además, se describe un caso de uso desarrollado por el Grupo de Digitalización del Centro de Fabricación Avanzada Aeronáutica (CFAA), donde se aplica RAG para optimizar el acceso a la documentación técnica de una máquina de brochado externo, facilitando la consulta de información dispersa y mejorando la eficiencia en las operaciones de mantenimiento y soporte técnico.

Problema de los LLMs puros

Conocimiento estático: limitado a datos de entrenamiento. Dominios específicos no cubiertos

Alucinaciones: generan información incorrecta con confianza

Sin fuentes: no citan de dónde viene la información

Contexto limitado: ventana de tokens finita

Actualización costosa: reentrenar es caro y lento

RAG: Solución

Recuperar documentos relevantes de una base de conocimiento

Incluir esos documentos en el contexto del prompt

El LLM genera respuesta basada en información recuperada

Beneficios: respuestas actualizadas, trazabilidad, menor alucinación

Arquitectura RAG básica

1. **Indexación**: procesar y almacenar documentos
2. **Recuperación**: buscar documentos relevantes para la query
3. **Generación**: LLM genera respuesta usando documentos recuperados

Pipeline offline (indexación) + online (recuperación + generación)

RAG vs Fine-tuning

Fine-tuning:

- Modifica pesos del modelo
- Costoso computacionalmente
- Difícil actualizar conocimiento

RAG:

- Modelo sin cambios
- Actualización fácil (añadir documentos)

Naive RAG (RAG Vectorial)

Enfoque más común y básico de RAG

Basado en embeddings vectoriales y similitud semántica

Pasos:

1. Chunking: dividir documentos en fragmentos
2. Embedding: convertir chunks a vectores
3. Almacenar en vector database
4. Query embedding + búsqueda por similitud

Naive RAG: Chunking

Dividir documentos largos en fragmentos manejables

Estrategias:

- Fixed-size: tamaño fijo (ej. 512 tokens)
- Sentence-based: por oraciones
- Paragraph-based: por párrafos
- Semantic chunking: dividir por cambios semánticos

Trade-off: chunks pequeños (precisos) vs grandes (contexto completo)

Naive RAG: Embeddings

Convertir texto a representación vectorial densa

Modelos populares:

- OpenAI text-embedding-3
- Sentence-BERT (SBERT)
- Cohere embeddings
- BGE (Beijing Academy of AI)

Vectores capturan significado semántico

Vector Databases

Bases de datos optimizadas para búsqueda vectorial

Algoritmos ANN (Approximate Nearest Neighbors):

- FAISS (Facebook)
- Pinecone (cloud)
- Weaviate
- Milvus
- Chroma

Naive RAG: Limitaciones

Pierde estructura y relaciones entre información

Chunks independientes sin conexión

Difícil responder preguntas que requieren múltiples saltos

No captura conocimiento implícito ni razonamiento

Sensible a calidad de embeddings y chunking

Graph RAG

RAG basado en grafos de conocimiento (Knowledge Graphs)

Mantiene estructura y relaciones entre entidades

Ventajas sobre Naive RAG:

- Preserva contexto y relaciones
- Razonamiento sobre múltiples saltos
- Conocimiento explícito y verificable
- Trazabilidad completa

Graph RAG: Arquitectura

1. Extraer entidades y relaciones de documentos
2. Construir Knowledge Graph (RDF/OWL)
3. Query: traducir pregunta a consulta estructurada (SPARQL)
4. Recuperar subgrafo relevante
5. Serializar grafo a texto para LLM
6. LLM genera respuesta usando grafo como contexto

Graph RAG con RDF/OWL

Knowledge Graphs semánticos con estándares W3C

Ventajas:

- Ontologías para definir estructura del dominio
- Razonamiento automático (OWL)
- Consultas complejas (SPARQL)
- Validación de datos (SHACL)
- Interoperabilidad (URIs, Linked Data)

Graph RAG: Extracción de conocimiento

Convertir texto no estructurado a triples RDF

Métodos:

- NER + Relation Extraction (tradicional)
- LLMs para extracción (GPT-4, etc.)
- Ontology-guided extraction
- Hybrid: combinar métodos

Resultado: texto → <sujeto, predicado, objeto>

Graph RAG: Ejemplo

Pregunta: "¿Qué ciudades están cerca de Bilbao?"

Proceso:

1. Identificar entidad: Bilbao
2. Query SPARQL: ciudades con distancia < 50km
3. Recuperar: Bilbao → cercaDe → Donostia, Vitoria
4. LLM genera: "Las ciudades cercanas son..."

Respuesta verificable con fuentes explícitas (URIs)

Graph RAG: Multi-hop reasoning

Responder preguntas que requieren múltiples saltos

"¿Dónde estudió el supervisor de Mikel?"

```
SELECT ?universidad WHERE {
    ex:Mikel ex:supervisadoPor ?supervisor .
    ?supervisor ex:estudióEn ?universidad
}
```

Graph RAG navega relaciones naturalmente

Hybrid RAG

Combinar múltiples enfoques de recuperación

Vector + Graph RAG:

- Vector search para similitud semántica
- Graph search para relaciones estructuradas
- Fusión de resultados (ranking)

Mejor de ambos mundos: flexibilidad + estructura

Otros tipos de RAG

SQL RAG: consultas sobre bases de datos relacionales

Document RAG: recuperación de documentos completos

Multi-modal RAG: imágenes, tablas, código

Conversational RAG: mantener contexto en diálogos

Self-RAG: modelo decide cuándo recuperar información

Advanced RAG: Reranking

Mejora la recuperación inicial con segundo paso de ranking

Proceso:

1. Recuperar top-100 candidatos (rápido, impreciso)
2. Reranker evalúa relevancia precisa (lento, preciso)
3. Seleccionar top-5 para LLM

Modelos: Cohere Rerank, Cross-encoders

Advanced RAG: Query expansion

Mejorar query del usuario antes de buscar

Técnicas:

- Reformulación con LLM
- Generar múltiples queries alternativas
- Desambiguación de términos
- Añadir sinónimos o contexto

Mejora recall y precisión

Advanced RAG: HyDE

Hypothetical Document Embeddings

Proceso:

1. Usuario hace pregunta
2. LLM genera respuesta hipotética
3. Buscar documentos similares a la respuesta hipotética
4. Generar respuesta final con docs recuperados

Mejora cuando query y documentos tienen diferente estilo

RAG: Evaluación

Métricas de recuperación:

- Precision@k: relevancia de top-k resultados
- Recall: fracción de docs relevantes recuperados
- MRR (Mean Reciprocal Rank)

Métricas de generación:

- Correctness: precisión factual
- Faithfulness: fidelidad a fuentes

RAG: Frameworks

LlamaIndex: framework RAG completo (Python)

LangChain: orquestación de LLMs y RAG

Haystack: NLP pipelines, RAG

txtai: embeddings y búsqueda semántica

Microsoft GraphRAG: Graph RAG con LLMs

Model Context Protocol (MCP)

Protocolo abierto de Anthropic para conectar LLMs con fuentes de datos

Objetivo: estandarizar integración de contexto externo

Componentes:

- **MCP Hosts**: aplicaciones con LLMs (Claude, IDEs)
- **MCP Clients**: mantienen conexiones con servidores
- **MCP Servers**: exponen recursos (datos, herramientas)

MCP: Características

Protocolo basado en JSON-RPC 2.0

Transporte: stdio, HTTP con SSE

Primitivas:

- **Resources**: datos que el LLM puede leer
- **Prompts**: plantillas reutilizables
- **Tools**: funciones que el LLM puede ejecutar
- **Sampling**: LLM puede solicitar inferencias recursivas

MCP Resources

Exponer datos para que LLM los consuma

Ejemplos:

- Archivos locales (filesystem)
- Bases de datos (SQL, RDF)
- APIs externas
- Documentos en la nube

URI scheme: file://, db://, http://, ...

MCP Tools

Funciones que el LLM puede invocar

Definición con JSON Schema para parámetros

Ejemplos:

- Ejecutar queries SPARQL
- Buscar en vector database
- Llamar APIs externas
- Operaciones CRUD en bases de datos

MCP: Ejemplo servidor

```
from mcp.server import Server
from mcp.types import Resource, Tool


server = Server("sparql-server")


@server.list_resources()
async def list_resources():
    return [Resource(uri="kg://museums", name="Museums KG")]
```

MCP + RAG

MCP estandariza cómo LLMs acceden a fuentes RAG

Servidor MCP puede exponer:

- Vector database como Resource
- SPARQL endpoint como Tool
- Búsqueda híbrida como Tool

Ventaja: múltiples aplicaciones LLM pueden usar mismo servidor

MCP: Adopción

Soportado por:

- Claude Desktop App (Anthropic)
- VS Code + extensiones
- IDEs modernos

Servidores disponibles:

- Filesystem, Git, PostgreSQL, Slack
- Google Drive, GitHub, GitLab

Skills (Capacidades de Agentes)

Skills: funcionalidades que agentes IA pueden ejecutar

Similar a MCP Tools pero más general

Tipos de skills:

- Information retrieval (RAG)
- Data processing (transformaciones)
- External actions (APIs, comandos)
- Reasoning (cálculos, lógica)

Skills: Composición

Skills se pueden componer para tareas complejas

Ejemplo: "Analiza ventas del último trimestre"

1. Skill: Consultar base de datos (SQL RAG)
2. Skill: Calcular estadísticas
3. Skill: Generar gráfico
4. Skill: Crear informe en PDF

Agente orquesta secuencia de skills

Skills + Knowledge Graphs

Skill: "Query Knowledge Graph"

- Input: pregunta en lenguaje natural
- Proceso: traducir a SPARQL
- Ejecutar query contra triple store
- Formatear resultados
- Output: respuesta estructurada

Combina NLP + razonamiento simbólico

Agentic RAG

RAG donde agente decide estrategia de recuperación

Agente tiene múltiples skills/tools:

- Vector search
- Graph traversal
- SQL queries
- Web search

Agente razona sobre qué tool usar y cuándo

Agentic RAG: ReAct

ReAct: Reasoning + Acting

Agente alterna razonamiento y acción:

1. Thought: "Necesito información sobre X"
2. Action: Ejecutar búsqueda
3. Observation: Resultado de búsqueda
4. Thought: "Ahora necesito Y"
5. Action: Consultar base de datos

RAG + Ontologías: Semantic RAG

Usar ontologías OWL para mejorar RAG

Ventajas:

- Query expansion con jerarquías (rdfs:subClassOf)
- Razonamiento automático (inferencias OWL)
- Validación de respuestas (SHACL)
- Desambiguación con definiciones formales

Ejemplo: buscar "mamíferos" recupera también "perros", "gatos"

RAG en producción: Desafíos

Latencia: recuperación + generación debe ser rápida

Escalabilidad: millones de documentos

Actualización: mantener índices sincronizados

Costes: embeddings, almacenamiento, llamadas LLM

Privacidad: datos sensibles en contexto

Calidad: evaluar y monitorear respuestas

RAG: Mejores prácticas

Chunking inteligente: preservar unidades semánticas

Metadata: enriquecer chunks con contexto

Hybrid search: combinar vector + keyword

Reranking: mejorar selección de documentos

Citation: incluir referencias en respuestas

Evaluation: medir calidad continuamente

RAG: Casos de uso

Customer support: responder con documentación actualizada

Legal/Compliance: consultas sobre regulaciones

Investigación científica: búsqueda en literatura

Enterprise search: búsqueda en datos corporativos

Educación: tutores IA con material del curso

Healthcare: asistentes con historiales médicos

Futuro de RAG

RAG multimodal: texto, imágenes, video, audio

Modelos nativos con RAG integrado

Retrieval más inteligente: comprensión profunda

Personalización: RAG adaptado al usuario

Verificación automática: fact-checking en tiempo real

Estandarización: protocolos como MCP

Recursos para aprender RAG

RAG paper original (Lewis et al.)

LlamaIndex documentation

Microsoft GraphRAG

Model Context Protocol

Self-RAG paper

Pinecone RAG guide